
GenoLeWM: An Action-Conditioned Latent World Model for Genomic Edits.

A Reproducible Pipeline and an Honest Negative Result

Abdelhamid Bakhta

<https://github.com/AbdelStark/GenoLeWM>

Abstract

A DNA foundation model scores a sequence, but it does not predict the *consequence of an explicit edit*: whether a learned latent transition operator can predict a post-edit embedding cheaply enough to support variant scoring, multi-edit roll-out, and edit planning is an open question. We present GenoLeWM, an action-conditioned Joint-Embedding Predictive Architecture that freezes Carbon-500M as a state encoder and trains a small cross-attention predictor $\hat{s}_{t+1} = g(s_t, a)$ to map a reference-window embedding and a discrete genomic edit action to the edited-window embedding, equipped with an unsupervised surprise score, a cross-entropy-method (CEM) latent planner, and a fully content-addressed train→eval→benchmark→replay pipeline. On the released v0.2.1-r1 checkpoint, the predictor’s multi-edit rollout reaches a cosine similarity of only 0.289 to the true post-edit state, which is 0.709 *below* a trivial “predict-no-change” baseline of ≈ 0.998 , and it trails Carbon-500M zero-shot AUROC by -0.1875 on ClinVar coding and -0.3125 on ClinVar non-coding. We argue this is a *structural* latent-residual trap rather than noise: a single edit barely moves a frozen 1024-dimensional embedding, so the identity map is already a strong baseline, and the frozen encoder was never shaped to expose edit directions. The evaluation slices ($N=16$ per ClinVar slice) cannot distinguish near-parity from chance, and the very freezing that delivers the “pay-Carbon-once” efficiency thesis is what removes the encoder’s ability to become edit-predictable. Our contribution is therefore a reproducible, checksum-receipted systems artifact and a falsifiable diagnosis, not a model that beats its encoder, together with a twelve-direction program for testing whether edit-conditioned latent world modeling is learnable at all against a near-frozen genomic encoder.

1 Introduction

DNA foundation models [7, 11, 14–16] answer a representational question: *what is the embedding, or the likelihood, of this sequence?* A different and largely unanswered question is *interventional*: given a reference genomic window and an explicit edit (a single-nucleotide variant, an insertion, or a deletion), can a model predict the *consequence* of that edit directly in representation space, cheaply enough to score variants, roll out multi-edit haplotypes, and plan edit sets? An affirmative answer would turn a static genomic encoder into an action-conditioned *world model* [8, 9] whose actions are discrete genomic edits.

GenoLeWM tests this question in its most economical form. We freeze a pretrained DNA encoder, Carbon-500M [10], and train only a small predictor: a reference window w_{ref} is encoded once to a state $s_t = \text{enc}(w_{\text{ref}}) \in \mathbb{R}^{1024}$, an edit action a is embedded by a lightweight action encoder, and a

cross-attention predictor estimates the post-edit state $\hat{s}_{t+1} = g(s_t, a)$, supervised against the frozen encoding $s_{t+1} = \text{enc}(w_{\text{alt}})$ of the edited window (fig. 1). Once $\hat{s}_{t+1}$ is available, three downstream operations (surprise scoring, autoregressive rollout, and CEM planning) run entirely in latent space and never re-invoke the encoder. This is the “pay-Carbon-once” design: amortize one expensive encode across many cheap latent queries.

The headline is negative, and we lead with it. GenoLeWM does *not* broadly beat the encoder it is built on. On the released v0.2.1-r1 benchmark, the predictor trails Carbon-500M zero-shot AUROC by -0.1875 (ClinVar coding) and -0.3125 (ClinVar non-coding), and its multi-edit rollout attains cosine ≈ 0.29 to the true post-edit state, roughly 0.70 *below* a trivial baseline that predicts no change at all (fig. 3). Rather than bury this, we treat it as the paper’s empirical core, because the mechanism behind it is informative: it is a *latent-residual baseline trap* that we believe is structural to the frozen-encoder regime, not a tuning failure. The same evidence package is content-addressed end to end: every number is bound to a model identity, a dataset snapshot, a commit, and a hardware string through checksum receipts, and the release toolchain re-renders its own report from those artifacts and refuses to pass unless the negative findings are present in them.

Contributions. This paper makes four contributions.

1. **A frozen-encoder, action-conditioned latent world model for genomic edits** (section 3): a discrete-edit action encoder and a cross-attention residual predictor over a frozen Carbon-500M state, with surprise scoring, KV-cached autoregressive rollout, and a CEM latent planner.
2. **A content-addressed train→eval→benchmark→replay pipeline** (section 4) in which a SHA-256-over-canonical-JSON scheme binds every artifact and output, and a verifier re-renders the release report and *structurally requires* the negative findings to be real before it will pass.
3. **A negative result with pre-registered baselines** (section 5) measured against two *a priori* naive baselines, Carbon-500M zero-shot likelihood for variant effect and a source-state “no-change” predictor for rollout fidelity, reported with their (small) sample sizes and quantization.
4. **A mechanistic diagnosis and research program** (sections 6 and 10): we attribute the negative result to a latent-residual trap compounded by frozen-encoder geometry and under-powered evaluation, and lay out twelve falsifiable directions to test whether edit-conditioned latent prediction is learnable against a near-frozen encoder.

What we do *not* claim. GenoLeWM is not a clinical, diagnostic, or decision-support tool; its outputs are research signals. We do not claim statistical significance for the single positive variant-effect delta we observe, a privacy or runtime-execution guarantee (the receipts certify *identity*, not correct execution), useful planning behavior, or that the efficiency thesis is demonstrated by our measurements. We make these boundaries precise throughout and collect them in section 7.

2 Related Work

GenoLeWM sits at the intersection of four research programs. DNA foundation models supply the encoder; variant-effect prediction supplies the evaluation and the foil; joint-embedding predictive architectures supply the objective and its principal failure mode; and latent world models supply the action-conditioned framing and the planning interface. The negative result is most legible against all four.

DNA and genomic foundation models: encoders, not edit predictors. A first wave ported masked- and causal-language modeling to nucleotides: DNABERT [11], the multi-species Nucleotide Transformer [7], the long-context HyenaDNA [14], the reverse-complement-equivariant state-space model Caduceus [16], and the genome-scale generative models Evo and Evo 2 [4, 15]. Their unifying property is that they output a *representation* or a *likelihood*; where they predict, they predict tokens, not the consequence of an intervention. Carbon-500M [10] is the instance GenoLeWM builds on, a permissively licensed, consumer-hardware-runnable decoder-only DNA model. We use it strictly as a *frozen* state encoder: pay once per reference window, never fine-tune in Phase 1. The distinction is concrete: these models answer “what is the representation of this sequence?”; GenoLeWM trains a small head to answer “given the frozen representation of a reference window and an explicit edit action, what is the representation of the edited window?” This is an

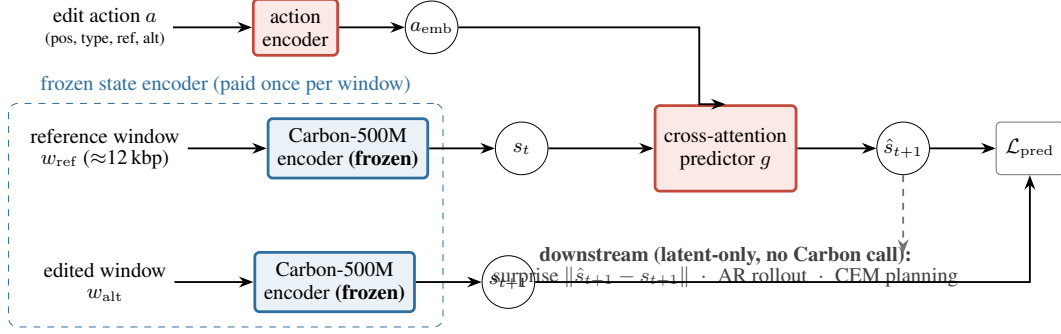


Figure 1: **GenoLeWM as an action-conditioned latent world model over genomic edits.** A frozen Carbon-500M DNA encoder maps a reference window w_{ref} to a state $s_t \in \mathbb{R}^{1024}$; a small trainable action encoder maps an explicit edit $a = (\text{pos, type, ref, alt})$ to a_{emb} ; and a cross-attention predictor g estimates the post-edit state $\hat{s}_{t+1} = g(s_t, a_{\text{emb}})$. Only the action encoder and predictor are trained ($\approx 40\text{M}$ params); the training target s_{t+1} is the frozen encoding of the edited window w_{alt} . Once $\hat{s}_{t+1}$ is available, every downstream use (surprise scoring, multi-edit rollout, CEM planning) operates entirely in latent space and never re-invokes Carbon.

edit-conditioned latent transition operator, with the edit as a *conditioning input* rather than a masked perturbation of the target.

Variation-effect prediction and its resources, and the precise sense in which we are not a clinical predictor. Our evaluation adopts the standard ecosystem: ClinVar clinical interpretations [13], gnomAD population frequencies [12] (which we repurpose as a per-context null model for surprise calibration), the TraitGym causal-regulatory benchmark [3], a BRCA2 saturation genome-editing functional assay, and AlphaMissense [5] as the high end of supervised missense prediction. GenoLeWM is not a clinical predictor, and the sense is precise along four axes. First, it is *label-free by construction*: surprise is an unsupervised by-product of predictor residual error, and no pathogenicity label enters training. Second, every output is a *signal, not a decision*, and we disclaim clinical use. Third, our baseline is the *encoder itself*, Carbon-500M zero-shot likelihood, because the scientific question is narrow and internal: does action-conditioning add evidence over the encoder it sits on? It is not “does GenoLeWM beat AlphaMissense?”, and conflating the two would be the central overclaim to avoid. Fourth, the honest finding is that GenoLeWM does not, yet, even beat Carbon-500M; so “not a clinical predictor” is an accurate scope description rather than modesty. We adopt the field’s benchmarks for comparability while declining the field’s claims.

JEPAs and anti-collapse SSL: the objective, and why freezing sidesteps collapse but springs a different trap. The objective lineage is the Joint-Embedding Predictive Architecture. I-JEPA [1] established that predicting in representation space, rather than reconstructing pixels, yields strong features; the genomic analogue is direct, since GenoLeWM predicts Carbon-500M-encoded window embeddings rather than nucleotides. The defining hazard of any predict-in-latent objective is *representation collapse*, and the SSL literature offers a menu of defenses: stop-gradient/EMA targets, variance-covariance regularizers, contrastive negatives, and distribution matching. The most recent of these, LeJEPA [2], replaces heuristics with an isotropic-Gaussian regularizer (via sketched normality testing), and GenoLeWM inherits its *form*: a Phase-2 closed-form KL between the batch-empirical Gaussian and $\mathcal{N}(0, I)$. The difference is that *in Phase 1 the encoder is frozen*. Frozen targets cannot be driven to a constant, so collapse is mechanically impossible and the Gaussian regularizer is monitored rather than trained. This is a legitimate simplification, but its price is the latent-residual baseline trap (section 6): with a fixed pretrained target manifold on which a single edit moves the embedding only slightly, the trivial predictor $\hat{s}_{t+1} := s_t$ becomes a *strong* baseline, not because it is good but because the quantity to be predicted is small relative to embedding scale. Methods that train the encoder jointly never face this fully; by freezing Carbon-500M, GenoLeWM forgoes that adaptation. This is exactly why we designed the rollout evaluation, *a priori*, with a naive source-state baseline.

Table 1: Where GenoLeWM sits. The load-bearing choice is freezing the encoder (collapse-immune by construction, but residual-trap-prone) with a *discrete genomic edit* as the action.

Method family	Learns encoder?	Anti-collapse	Action	Planning
DNA FMs [7, 11, 14, 15]	yes (pretrain)	n/a	none	none
Supervised VEP [5]	task-tuned	n/a	none	none
JEPA / LeJEPA [1, 2]	yes, jointly	EMA / Gaussian reg.	n/a	none
Latent world models [6, 8, 9]	yes	recon./reward	continuous	imagination / MPC
GenoLeWM (this work)	no (frozen)	n/a (frozen)	discrete edit	CEM / MPC

Latent world models and planning: a world model whose action is a discrete genomic edit.

The architectural lineage runs from Ha and Schmidhuber’s World Models [8] through the Dreamer family [9] to sampling-based latent model-predictive control. GenoLeWM’s planner is the cross-entropy method in latent space with MPC replanning, the same family as PETS [6]. The distinctive move is the *type of action*: in all of the above the action is a continuous control vector, whereas in GenoLeWM the action is a discrete genomic edit (position, type, ref, alt) over {SNV, INS, DEL, MNV, INDEL}, so planning is combinatorial search over a discrete edit space, scored by latent distance after predictor rollout. Honestly read, GenoLeWM satisfies the *architectural* definition of a world model (it predicts the next latent state given an action and exposes a planner) without yet the *behavioral* one: our planning demonstration records execution, not useful planning. Dreamer and MPC methods earn the title through demonstrated control; we concede we have earned it structurally but not empirically.

Table 1 summarizes the positioning.

3 Method

GenoLeWM has three trained-or-frozen components (a frozen state encoder, a trainable action encoder, and a trainable predictor) and three latent-space operations that reuse them: surprise scoring, rollout, and planning. We describe the released v0.2.1-r1 configuration; where the running configuration diverges from the design RFCs on a load-bearing hyperparameter, we report the *released* value and tabulate the divergence in table 2.

3.1 State encoder (frozen)

The encoder is Carbon-500M, a 500M-parameter decoder-only DNA language model (LLaMA-style, hidden size 1024), loaded at a pinned revision in bf16 and never updated in Phase 1. A reference window of 12,288 bp (= 2,048 six-mer tokens), wrapped in Carbon-500M’s custom tokenizer markers, is passed through the model; we take hidden states at layer $L=20$ and pool them by a *centered mean* over a ± 8 -token window (± 48 bp) around the edit locus, then L_2 -normalize:

$$s_t = \frac{1}{\|\cdot\|_2} \text{mean}\left(H_L[c-r : c+r+1]\right), \quad c = \lfloor \text{locus}/6 \rfloor, \quad r = 8, \quad H_L \in \mathbb{R}^{2048 \times 1024}, \quad (1)$$

giving $s_t \in \mathbb{R}^{1024}$. Reference-window embeddings are cached in content-addressed Parquet shards indexed by a SQLite index keyed on the window hash, encoder hash, layer, pool type, pool radius, and dtype; edited windows are encoded on demand. The ± 48 bp pooling radius is far tighter than the RFC’s ± 1536 bp default and is a deliberate locality choice; we flag it because it determines how much sequence context the state actually summarizes.

3.2 Action encoder (trainable)

An edit is represented as an EditSpec (absolute VCF coordinates chrom, pos, ref, alt) or its window-relative form RelEdit, with edit type derived from the (ref, alt) lengths into {SNV, INS, DEL, MNV, INDEL} and a 16 bp cap on either allele. The action encoder maps an edit to $a_{\text{emb}} \in \mathbb{R}^{64}$ through four sub-encoders, namely a base-pair-resolution sinusoidal position embedding ($\mathbb{R}^{128}$), a learned type embedding ($\mathbb{R}^{64}$), and a *shared* small Transformer (“SeqMicroEncoder”) applied to the reference and alternate alleles ($\mathbb{R}^{256}$ each); these are concatenated to $\mathbb{R}^{704}$ and projected by a two-layer MLP with GELU and LayerNorm. The sinusoidal schedule uses the window length as its frequency base rather than the conventional 10,000.

3.3 Predictor (trainable, residual)

The predictor g is a cross-attention Transformer over the token sequence $[s_t; a_1; \dots; a_K]$, alternating state $\leftrightarrow$ action cross-attention blocks (six, in the released config) with self-attention blocks (two), hidden dimension 768, eight heads, GELU, pre-LayerNorm, learned token-type and step-position embeddings, and a causal mask so each action attends to the state and to earlier actions. The read-out is *residual* and L_2 -normalized, with the final MLP layer zero-initialized so that the predictor is the identity at initialization:

$$\hat{s}_{t+1} = \frac{s_t + \text{MLP}(g(s_t, a_{\text{emb}}))}{\|s_t + \text{MLP}(g(s_t, a_{\text{emb}}))\|_2}. \quad (2)$$

Equation (2) is central to the paper: *at initialization*, $\hat{s}_{t+1} = s_t$ is exactly the “predict-no-change” baseline, and training must move the prediction away from it by learning the small residual $\Delta = s_{t+1} - s_t$. An autoregressive wrapper (ARPredictor) unrolls the predictor step by step with KV-caching for multi-edit rollout and planning, upcasting the output MLP to fp32 beyond horizon 20.

3.4 Training objective (Phase 1, frozen encoder)

With the encoder frozen, only the prediction loss is trained:

$$\mathcal{L}_{\text{pred}}(\hat{s}_{t+1}, s_{t+1}) = \alpha (1 - \cos(\hat{s}_{t+1}, s_{t+1})) + \beta \frac{\|\hat{s}_{t+1} - s_{t+1}\|_2^2}{d}, \quad \alpha=1.0, \beta=0.1, d=1024, \quad (3)$$

averaged uniformly over rollout steps with a mask. The LeJEPa isotropic-Gaussian regularizer $\mathcal{L}_{\text{reg}} = \frac{1}{2}(\|\mu\|^2 + \text{tr } \Sigma - \log \det(\Sigma + \epsilon I) - d)$ is computed for *monitoring only* in Phase 1, because a frozen target cannot collapse, and becomes an active term ($\gamma=0.5$) only when the encoder is LoRA-adapted in a future Phase 2. We optimize with AdamW ($\beta_2=0.95$, weight decay 0.1, gradient clip 1.0) under a warmup-stable-decay schedule, and log seven collapse diagnostics (predicted variance per dimension, pairwise predicted distance, predicted-target correlation, the monitored KL, *etc.*) every 500 steps with explicit alert thresholds. The released run uses seed 104729, 20,000 steps, and a deterministic mode.

3.5 Surprise scoring

The raw surprise of a variant is the predictor residual against the true post-edit state, $\sigma_{\text{raw}}(v) = \|g(s_t, a_v) - \text{enc}(\text{apply}(v, w_{\text{ref}}))\|_2$. To make it comparable across genomic context, we calibrate it per context bucket $b(v) = \text{region} | \text{gc} | \text{repeat}$ ($11 \times 3 \times 5 = 165$ buckets) against an empirical CDF estimated from up to 10,000 common gnomAD variants per bucket, with a backoff chain when a bucket is sparse, yielding $\sigma_{\text{cal}}(v) = F_{b(v)}(\sigma_{\text{raw}}(v)) \in [0, 1]$. Note that σ is the variant-effect score we evaluate; it is unsupervised and label-free.

3.6 Latent planning

Given an initial state and a target latent, the CEM planner searches over K -edit sequences by sampling a factored proposal over edit type, position, and bases, evaluating each candidate by latent distance after a K -step predictor rollout (with optional edit-cost regularization), and refitting the proposal to the elite set with smoothing. It uses defaults of horizon 5, five iterations, 1024 samples, and 64 elites. Carbon-500M is called once to encode the initial and target windows; the entire inner search runs in latent space and never re-invokes the encoder.

3.7 The efficiency thesis and its honest scope

The design rationale is amortization: pay one Carbon-500M forward pass per reference window, then answer many downstream queries cheaply with the $\sim 40\text{M}$ -parameter predictor. This argument is *correct for rollout and planning*, which make zero further encoder calls after the initial encode. It is only *partial* for single-variant scoring, which by construction requires a second Carbon-500M call on the *edited* window to produce the target s_{t+1} ; with a warm reference cache this halves the encoder cost, but it does not eliminate it. As we show in section 5.3, the released latency benchmark exercises none of these amortized regimes, so the efficiency thesis remains architecturally sound but operationally unverified.

Table 2: **Released configuration vs. design RFCs** for load-bearing hyperparameters. The released v0.2.1-r1 values are canonical for every number in this paper; the RFC values are the (aspirational) specification. Divergences change what the state summarizes and the predictor’s capacity, and any reader reproducing the work must use the released column.

Hyperparameter	Released (v0.2.1-r1)	Design RFC
Encoder pooling layer L	20	final (−1)
Pool radius r	8 tok (± 48 bp)	256 tok (± 1536 bp)
State dim d_{state}	1024	1024 (package default 512, overridden)
Action dim d_{action}	64	512
Predictor cross + self blocks	6 + 2	4 + 2
Predictor hidden dim d_{hidden}	768	1024
FFN intermediate dim	768 (i.e., $1\times$)	2048 ($2\times$)
Warmup steps	1000	2000
Weight decay	0.1	0.05
Physical batch size	8	256 effective
Edit-source mix (gnomAD/synSNV/synINDEL/ClinVar)	3:3:1:1	4:3:2:1

4 Systems and Reproducibility

The most defensible positive contribution of this work is infrastructural: every number we report is content-addressed and replayable, and the release toolchain is built so that the negative findings cannot be quietly edited away.

Content-addressed provenance. A canonical-JSON serialization (sorted keys, rejected NaN/Inf, UTF-8) feeds a SHA-256 scheme in which the *model identity* is literally the hash of the model manifest, which in turn hashes every artifact file. Each scoring call emits a checksum receipt binding the model id, an input commitment (a hash over the reference window, edit spec, pooling and dtype configuration), the output, an output commitment, the calibration hash, and runtime metadata. The runtime verifies all manifest artifact hashes at construction and refuses absolute or traversing paths. We are explicit that these receipts certify *integrity and identity*, *not attestation*: they prove which artifacts and outputs were produced, not that execution was correct; there is no trusted-execution or proof-of-computation layer.

Evidence-bound benchmark suite. The benchmark suite executes a fixed step graph (VEP scoring, the Carbon-500M zero-shot baseline, VEP evaluation, rollout-state generation and evaluation, aggregation, and a readiness gate), recording for each step the content identity (path, SHA-256, size) of its outputs and gating the whole run on every step passing. A readiness report then requires all nine benchmark rows to be present and carry their observed values, baseline deltas, and a non-empty list of negative findings; the evaluated release reports 28 verified suite outputs.

A verifier that requires the negatives to be real. The release paper toolchain re-renders the machine-readable manuscript from the artifacts at verification time and rejects any text that does not match: no stale tables, no placeholder tokens, no mutated planning evidence. More pointedly, it *hard-codes* the negative findings as pass conditions: the readiness artifacts must show a negative ClinVar non-coding AUROC delta, a negative BRCA2 Spearman delta, rollout weakness on both splits, and a $K=20$ rollout speedup below the $5\times$ target, with the autoregressive-speed row explicitly marked rescoped under an accepted scope decision. In other words, the pipeline cannot certify itself unless the negative result is present in the evidence that drives it. This inverts the usual incentive of result-reporting infrastructure and is, we argue, the right shape for a reproducibility artifact whose honest conclusion is a negative one.

On-device posture. The runtime additionally enforces a fail-closed network guard during inference and ships local-first importers for consumer personal-genome formats, with a redaction filter that drops DNA-like strings and a deny-list of personal fields at the logging boundary. We describe these as engineering choices for local execution; we explicitly do *not* present them as a privacy guarantee.

Table 3: **Variant-effect prediction (v0.2.1-r1)**. Binary ClinVar slices report accuracy / AUROC / average precision (AP) / balanced accuracy; BRCA2 saturation and TraitGym Mendelian report Spearman ρ against continuous functional/causal labels. The baseline is the same frozen Carbon-500M encoder scored zero-shot. Δ is GenoLeWM – Carbon (positive favours GenoLeWM). Bold marks the better value per row. GenoLeWM shows a positive delta only on coding-ClinVar accuracy/balanced accuracy (+0.0625=1/16, within quantization) and a TraitGym tie near zero; it trails on every pathogenicity-bearing ranking metric.

Benchmark	Metric	GenoLeWM	Carbon 0-shot	Δ
ClinVar coding	Accuracy $\uparrow$	0.7500	0.6875	+0.0625
	AUROC $\uparrow$	0.7344	0.9219	−0.1875
	AP $\uparrow$	0.8530	0.9519	−0.0989
	Balanced acc. $\uparrow$	0.7500	0.6875	+0.0625
ClinVar non-coding	Accuracy $\uparrow$	0.4375	0.6875	−0.2500
	AUROC $\uparrow$	0.5625	0.8750	−0.3125
	AP $\uparrow$	0.6055	0.9144	−0.3090
	Balanced acc. $\uparrow$	0.4375	0.6875	−0.2500
BRCA2 saturation	Spearman ρ $\uparrow$	0.1492	0.4769	−0.3277
TraitGym Mendelian	Spearman ρ $\uparrow$	−0.0280	−0.0839	+0.0559

5 Experiments

Setup. All results are from a single released identity (table 6): model `geno-lewm-v0.2.1-r1`, dataset snapshot `geno-lewm-data-v0.2.1-r1`, commit `d9b06815`, evaluated on one NVIDIA H200. There are six evaluation slices across three tracks, with the sample sizes shown in-line below. Two baselines are fixed *a priori*: for variant effect, Carbon-500M zero-shot, scored as the log-likelihood difference $\log P(w_{\text{ref}}) - \log P(w_{\text{alt}})$ (higher is more pathogenic); for rollout fidelity, a source-state predictor that returns s_t unchanged.

5.1 Variant-effect prediction

Table 3 and fig. 2 report the variant-effect track. On the two binary ClinVar slices ($N=16$ each) and BRCA2 saturation ($N=32$), GenoLeWM trails Carbon-500M zero-shot on every ranking metric: ClinVar coding AUROC 0.734 ($\Delta = -0.1875$), ClinVar non-coding AUROC 0.563 ($\Delta = -0.3125$), and BRCA2 Spearman 0.149 ($\Delta = -0.328$). The only positive deltas are ClinVar coding accuracy and balanced accuracy (+0.0625 each), and 0.0625 is exactly 1/16, i.e., a single one of sixteen variants, well within the slice’s quantization; we decline to call it a win. The TraitGym “win” (+0.056, $N=32$) is a difference between two Spearman values indistinguishable from zero (−0.028 vs. −0.084). Because every ClinVar metric is quantized in steps of 1/16, bootstrap confidence intervals over these slices are not meaningful, and we frame the result accordingly: on slices this size, GenoLeWM shows *no detectable improvement* over its encoder and shows *detectable deficits* only where the deltas exceed the implied resolution.

5.2 Latent rollout fidelity

The rollout track (table 4, fig. 3) is the paper’s clearest signal. Over two multi-edit settings ($N=8$ each), the predictor’s final state attains mean cosine 0.289 (phased haplotypes) and 0.302 (synthetic edit chains) to the true post-edit state. The trivial source-state baseline, which predicts no change, attains 0.998 and 0.991, because a single edit barely perturbs the 1024-dimensional frozen embedding. The predictor is thus 0.709 and 0.690 cosine *below* the do-nothing baseline, and farther in L_2 (+31.2, +25.6): the learned map is a *worse-than-identity distortion* rather than an imperfect mover toward s_{t+1} . Recall@ k is 1.0 for *both* the predictor and the baseline, so it does not discriminate at these sample sizes and we do not report it as a positive.

5.3 Inference efficiency

Table 5 and fig. 4 report the released efficiency measurement, and require care to interpret honestly. The benchmark times a cold `geno-lewm-score` subprocess end to end; the median single-variant

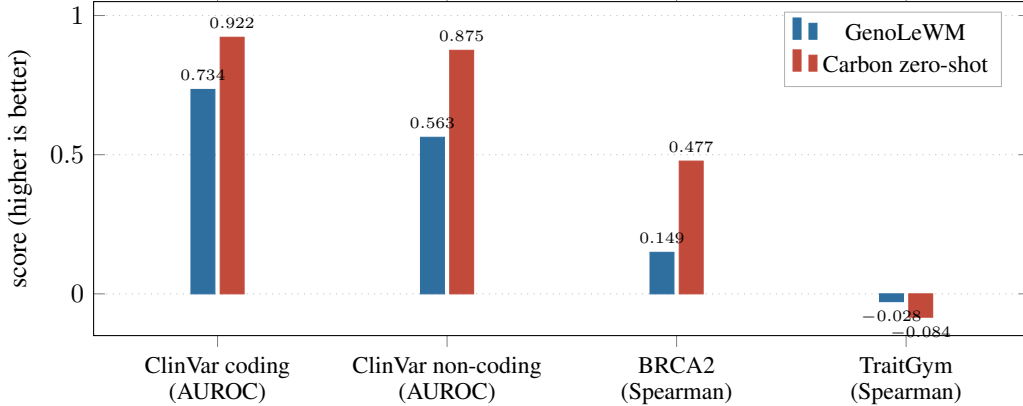


Figure 2: **Variant-effect prediction: GenoLeWM trails the Carbon zero-shot baseline on the pathogenicity-bearing slices.** Bars are the released v0.2.1 values; Carbon values are recovered as $\text{GenoLeWM} - \Delta$. GenoLeWM is worse on ClinVar coding/non-coding AUROC and BRCA2 saturation Spearman; the only “win” (TraitGym) is between two scores indistinguishable from zero. Slices are small (metrics quantized in steps of $0.0625=1/16$, consistent with ≈ 16 evaluated variants per ClinVar slice); see §7.

Table 4: **Latent rollout fidelity (v0.2.1-r1).** Mean cosine similarity ($\uparrow$) and mean L_2 distance ($\downarrow$) between the predicted final state $\hat{s}_{t+1}$ and the true encoded state s_{t+1} , over two multi-edit settings. The baseline predicts no change ($\hat{s}_{t+1}=s_t$). The do-nothing baseline dominates on both fidelity metrics, and Recall@4 is saturated at 1.0 for *both* the model and the baseline, so it does not discriminate. This is the latent-residual baseline trap (fig. 3).

Setting	Cosine to $s_{t+1} \uparrow$			L_2 to $s_{t+1} \downarrow$		
	GenoLeWM	Source-state	Δ	GenoLeWM	Source-state	Δ
Phased haplotypes	0.2889	0.9978	-0.7090	33.32	2.13	+31.19
Synthetic edit chains	0.3016	0.9912	-0.6896	28.80	3.17	+25.64

latency is 115,262.94 ms (≈ 115 s), throughput is 0.31 variants/s, and peak memory is 1.83 GiB. This number does not contradict the architecture so much as fail to test it: the subprocess pays interpreter startup, ~ 1 GiB of Carbon-500M model loading, and two Carbon-500M forward passes (reference and edited windows), and the benchmark self-documents that its wall-clock “includes CLI startup and artifact loading overhead.” The ~ 160 ms of genuine Carbon-500M compute is dwarfed by loading. Three regimes are relevant (fig. 4): the *measured* cold-subprocess regime; a *warm-cache, in-process* regime that the efficiency thesis targets (one Carbon-500M call, ~ 95 ms total, of which the predictor adds ~ 1 ms); and a *latent-only rollout/planning* regime (zero further Carbon-500M calls). Only the first is measured. We therefore claim neither that GenoLeWM is slow per inference nor that it is fast versus Carbon-500M; the apples-to-apples comparison (warm-cache, in-process) has not been made, and we mark it open.

5.4 Autoregressive rollout speed

KV-caching the action projections speeds autoregressive rollout by $2.41\times$ at $K=5$ (meeting its local $2\times$ target) and $2.47\times$ at $K=20$ (missing the $5\times$ target; fig. 5). The $K=20$ miss remains open and was explicitly rescoped rather than closed. We caution that this micro-benchmark runs on toy synthetic dimensions ($d_{\text{state}}=64$, CPU, fp32), so the ratios are not predictive of the Carbon-scale model on a GPU; they establish that the cache helps, not by how much in production.

5.5 Planning

The released planning demonstration loads the checkpoint through the manifest-backed runtime and runs CEM against a synthetic multi-SNV target, terminating on patience after 384 evaluations with a

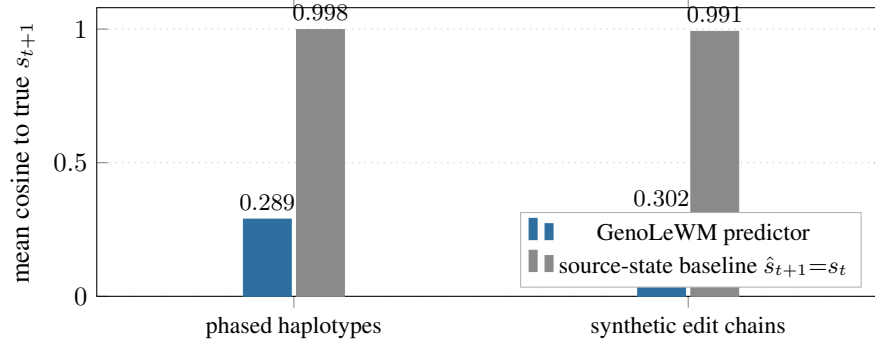


Figure 3: **The latent-residual baseline trap.** A trivial baseline that predicts *no change* ($\hat{s}_{t+1}=s_t$) attains cosine ≈ 0.99 to the true post-edit state, because a single SNV barely perturbs a 1024-dimensional frozen Carbon embedding. The trained predictor’s outputs sit at cosine ≈ 0.29 – 0.30 , far below the do-nothing baseline (deltas -0.709 and -0.690). This is the central diagnostic for why edit-conditioned latent prediction is hard in this regime (§6).

Table 5: **Measured efficiency (v0.2.1-r1, single NVIDIA H200).** The released single-variant score path is dominated by two frozen Carbon-500M forward passes under a cold cache and process start, so per-variant latency is ≈ 115 s; the architectural amortization thesis (fig. 1) is *not* exercised by this naive path (§5.3). The KV-cached autoregressive rollout speedup is the only place latent-only cheapness appears, and it remains below the $5\times$ target at $K=20$.

Quantity	Value
Single-variant latency (cold, 2 Carbon passes)	115,262.94 ms (≈ 115 s)
Batched throughput	0.3095 variants/s
Peak memory	1,966,149,632 B (≈ 1.83 GiB)
AR rollout speedup, $K=5$	$2.41\times$ (local $2\times$ target met)
AR rollout speedup, $K=20$	$2.47\times$ (RFC-0004 $5\times$ target <i>not</i> met, #42 open)

best latent distance of 23.66 in ≈ 15.3 s. It establishes that the planning path executes from released artifacts; it does *not* demonstrate useful planning behavior. The non-zero terminal distance and patience stop are recorded as evidence of integration, not capability, and the target is a synthetic proxy rather than a learned objective.

6 Discussion: Why the Negative Result Occurs

We argue the negative result is over-determined rather than accidental, and that low prediction loss is *not* evidence of a learned edit-transition operator. Four interacting causes account for it.

The latent-residual baseline trap (primary cause). A single SNV inside a 12 kbp window, mean-pooled over a frozen 1024-dimensional embedding, perturbs that embedding only slightly, so $s_{t+1} \approx s_t$ and $\cos(s_t, s_{t+1})$ is near one. The copy map $\hat{s}_{t+1} := s_t$ therefore captures most of the achievable cosine for free, and the predictor must learn only the small residual $\Delta = s_{t+1} - s_t$, which is low in norm, high in relative variance, and dominated by the bulk s_t direction. Because the predictor is residual with an identity initialization (eq. (2)), gradient descent starts *at* the copy solution and is attracted to it. The rollout evidence is exactly what this predicts: a predictor that genuinely learned Δ would beat copy, whereas one that learned a *distorted* copy sits below it, which is what we observe (-0.709 , -0.690 cosine). Phase-1 collapse-immunity is the *cause* here, not a defense: the Gaussian regularizer guards against *encoder* collapse, and nothing in Phase 1 guards against *functional collapse of the predictor onto identity*.

Representation geometry. For action-conditioning to be learnable, the map $(s_t, a_v) \mapsto \Delta$ must be *exposed* by the encoder’s geometry, so that similar edits produce similar, low-dimensional, roughly linear displacements. There is no guarantee a frozen model trained for sequence likelihood organizes

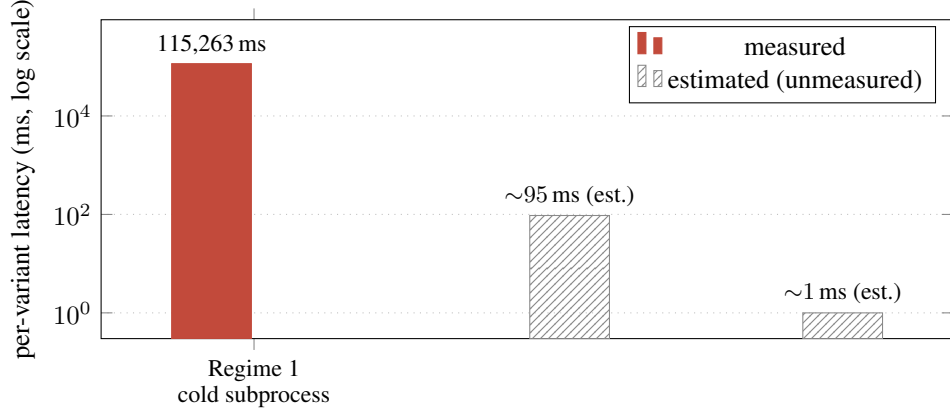


Figure 4: **The efficiency thesis is architecturally sound but unmeasured in the regime it describes.** Only *Regime 1*, a cold geno-lewm-score subprocess (interpreter start, ~1 GiB Carbon load, two Carbon forward passes), is measured, at 115,262.94 ms; the ~160 ms of actual Carbon compute is dwarfed by process start and model loading. The “pay-Carbon-once” thesis applies to *Regime 2* (warm reference cache: one Carbon call on the edited window, in-process) and *Regime 3* (latent-only rollout/planning: zero further Carbon calls), both *unmeasured* on this checkpoint; the hatched bars are order-of-magnitude estimates, not results (§5.3). The 115 s figure is therefore evidence of cold-start cost, neither evidence that GenoLeWM is slow per inference nor that it is fast versus Carbon.

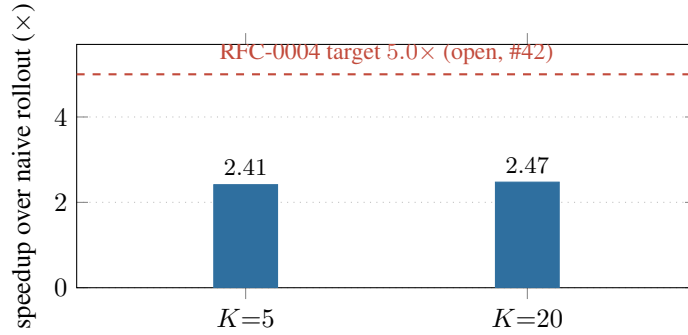


Figure 5: **KV-cached autoregressive rollout speedup.** Cached ARPredictor rollout is $2.41\times$ ($K=5$) and $2.47\times$ ($K=20$) faster than naive repeated one-step prediction. The gain is real but short of the $5\times$ target at $K=20$, which remains open and was explicitly rescoped. Measured on toy synthetic dimensions ($d_{\text{state}}=64$, CPU, fp32); not Carbon-scale.

its space this way; Δ may be near-isotropic noise of magnitude comparable to the encoder’s regional variance, nonlinear, or entangled with nuisance directions. A direct test is a linear-versus-MLP probe gap for decoding Δ , with Δ -norm compared against regional encoder variance (section 10, D3).

Tiny evaluation slices and absence of power. ClinVar slices of $N=16$ quantize every metric to multiples of $1/16$, so a one-to-two-point AUROC difference is within the resolution and bootstrap intervals are not meaningful. The honest statement is not “GenoLeWM trails Carbon-500M on every row with confidence” but “on slices this size, GenoLeWM shows no detectable improvement and shows detectable deficits only where deltas exceed the implied resolution.”

Frozen encoder may not expose an edit-linear latent (structural cause). Synthesizing the above: the encoder was optimized for the wrong invariance (what the sequence *is*, not how an edit moves it); the predictor has no leverage to reshape it; and so *the efficiency thesis and the learnability thesis are in tension*: freezing Carbon-500M enables “pay once, query many” but removes the encoder’s ability to become edit-predictable. The reading we favor is that the very choice that buys

efficiency may be the one that costs accuracy, and that the natural remedy (Phase-2 LoRA under the Gaussian regularizer) reintroduces exactly the collapse risk that Phase 1 was designed to avoid. The result is not that edit-conditioned latent world modeling is impossible, but that it is not learnable against a fully frozen, edit-agnostic encoder on under-powered slices. That is a falsifiable claim.

7 Limitations

We collect the boundaries that constrain every claim. (i) *Statistical power*: ClinVar slices are $N=16$ (metrics quantized at $1/16$) and rollout slices are $N=8$; no result here carries a meaningful confidence interval, and the lone positive delta is within quantization. (ii) *Coverage*: placed-window training is effectively single-chromosome (chr22), and the chr21 holdout specified in the data RFC is not confirmed in the released first-experiment configuration. (iii) *Configuration drift*: the released model diverges from the design RFCs on load-bearing hyperparameters (table 2), most consequentially the ± 48 bp pooling radius and the residual predictor depth, so the evaluated system is one specific point, not the RFC’s intended design. (iv) *Efficiency*: no warm-cache or in-process latency was measured, so the efficiency thesis is untested in the regime it describes; the autoregressive speedups are on toy dimensions. (v) *Planning and calibration*: the planner is evaluated on a synthetic, non-learned proxy and does not demonstrate useful planning; the surprise calibration is not validated on a held-out population. (vi) *Receipts*: checksum receipts certify artifact and output identity, not correct execution or any privacy property. (vii) *Maturity*: all governing RFCs remain in Draft status.

8 Negative Findings

For directness, we enumerate the negative findings as first-class results, each tied to its number and the artifact that carries it. GenoLeWM trails Carbon-500M zero-shot on ClinVar non-coding AUROC ($\Delta = -0.3125$) and on BRCA2 saturation Spearman ($\Delta = -0.328$), and on ClinVar coding AUROC/AP as well; its only positive ClinVar delta ($+0.0625$) is one variant within quantization. Its multi-edit rollout is below the trivial source-state baseline on both splits (cosine $\Delta = -0.709$ and -0.690). Its $K=20$ autoregressive rollout speedup ($2.47\times$) misses the $5\times$ target, which remains open. Its planning demonstration stops on patience with a non-zero best distance and does not prove useful planning behavior. And its released single-variant latency (≈ 115 s) reflects cold-process model loading rather than per-inference compute, leaving the efficiency thesis unverified. These are the substance of the paper, and the release verifier structurally requires them to be present in the evidence (section 4).

9 Reproducibility

Every reported value is bound to the identities in table 6 through the content-addressed receipt chain of section 4, and the release report (and its machine-rendered companion manuscript) re-derives from those artifacts. A reader can verify the artifact graph (hashes, sizes, the readiness gate, and that the negative findings are present and structurally required) without re-running the model. Reproducing the *numbers* from scratch additionally requires the released checkpoint and the dataset snapshot, the pinned Carbon-500M revision, and an equivalent GPU; the configuration in table 2 is the canonical recipe, and the deterministic training mode is available at a throughput cost. We note as an open reproducibility gap that the autoregressive-speed scope artifact was not persisted in the repository checkout, and that repeated-run bit-exactness is listed as future work.

10 Future Work

The diagnosis in section 6 organizes a falsifiable program; we sketch twelve directions as a decision tree, each with a hypothesis, a smallest falsifying experiment, and the baseline it must beat. **Attack the cause:** (D1) lightly LoRA-adapt Carbon-500M under the Gaussian regularizer so Δ becomes linearly extractable, with collapse ($KL > 10$) or persistent sub-copy rollout as the failure signal; (D2) add an edit-contrastive / whitened-residual objective to kill the identity attractor; (D3) probe whether Δ is even linearly accessible in frozen Carbon-500M (linear-vs-MLP gap, Δ -norm vs regional vari-

ance), the single highest-information experiment; (D4) curriculum on high-displacement edits (non-sense, splice) where the copy baseline is weak. **Fix measurement:** (D5) power analysis and slices sized to detect a two-point AUROC effect; (D6) stronger baselines (linear $s_t \rightarrow \Delta$, k -NN-retrieved Δ , Carbon-500M $\Delta \log \text{Lik}$); (D7) repeat Phase 1 across alternative frozen encoders (HyenaDNA, Caduceus, Evo 2) and correlate probe linearity with rollout margin; (D8) order-scrambled rollout controls for genuine edit composition. **Earn deployment/scale claims:** (D9) implement and benchmark the reference cache to test the efficiency thesis honestly and close or document the $5\times$ gap; (D10) distill into a sub-100M student and report the accuracy–latency Pareto; (D11) fit scaling laws for margin-over-copy-baseline against predictor size, edit count, and encoder scale; (D12) test whether surprise carries signal orthogonal to Carbon-500M $\Delta \log \text{Lik}$ and retire it honestly if not. The unifying question, *is edit-conditioned latent prediction learnable against a near-frozen genomic encoder, and at what compute?*, is the paper’s most valuable contribution to future work.

11 Conclusion

We built an action-conditioned latent world model for genomic edits on top of a frozen DNA foundation model, wrapped it in a content-addressed train→eval→benchmark→replay pipeline whose verifier requires its own negative findings to be real, and reported honestly that the learned predictor does not beat either the encoder’s zero-shot baseline or a trivial predict-no-change baseline on the released, under-powered evaluation. We traced this to a latent-residual trap that we argue is structural to the frozen-encoder regime: freezing the encoder eliminates representation collapse and enables the efficiency thesis, but in doing so makes the identity map a competitive baseline and leaves the residual hard to extract. The contribution is a reproducible artifact and a falsifiable diagnosis, not a method that wins, and the open question of whether edit-conditioned latent prediction is learnable against a near-frozen encoder is, we hope, worth more than a premature claim that it already is.

Acknowledgments

We thank the maintainers of the open genomic-modeling and self-supervised-learning ecosystems whose models, benchmarks, and methods this study builds on and measures against. Portions of the evidence-gathering, citation verification, and manuscript drafting were assisted by AI research tooling; all reported numbers derive from the content-addressed release artifacts and were checked against them.

References

- [1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, 2023. URL <https://arxiv.org/abs/2301.08243>.
- [2] Randall Balestriero and Yann LeCun. LeJEPa: Provable and scalable self-supervised learning without the heuristics. *arXiv preprint arXiv:2511.08544*, 2025. URL <https://arxiv.org/abs/2511.08544>.
- [3] Gonzalo Benegas, Gökçen Eraslan, and Yun S. Song. Benchmarking DNA sequence models for causal regulatory variant prediction in human genetics. *bioRxiv*, 2025. doi: 10.1101/2025.02.11.637758. URL <https://doi.org/10.1101/2025.02.11.637758>. TraitGym benchmark.
- [4] Garyk Brixi, Matthew G. Durrant, Jerome Ku, Michael Poli, et al. Genome modeling and design across all domains of life with Evo 2. *bioRxiv*, 2025. doi: 10.1101/2025.02.18.638918. URL <https://doi.org/10.1101/2025.02.18.638918>.
- [5] Jun Cheng, Guido Novati, Joshua Pan, Clare Bycroft, Akvilė Žemgulytė, Taylor Applebaum, Alexander Pritzel, Lai Hong Wong, Michal Zielinski, Tobias Sargeant, Rosalia G. Schneider, Andrew W. Senior, John Jumper, Demis Hassabis, Pushmeet Kohli, and Žiga Avsec. Accurate proteome-wide missense variant effect prediction with AlphaMissense. *Science*, 381(6664):eadg7492, 2023. doi: 10.1126/science.adg7492. URL <https://doi.org/10.1126/science.adg7492>.
- [6] Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. URL <https://arxiv.org/abs/1805.12114>. Introduces PETS; cross-entropy method (CEM) for model-predictive control.

- [7] Hugo Dalla-Torre, Liam Gonzalez, Javier Mendoza-Revilla, Nicolas Lopez Carranza, Adam Henryk Grzywaczewski, Francesco Oteri, Christian Dallago, Evan Trop, Bernardo P. de Almeida, Hassan Sirelkhatim, et al. Nucleotide transformer: building and evaluating robust foundation models for human genomics. *Nature Methods*, 22(2):287–297, 2025. doi: 10.1038/s41592-024-02523-z. URL <https://doi.org/10.1038/s41592-024-02523-z>.
- [8] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018. doi: 10.5281/zenodo.1207631. URL <https://arxiv.org/abs/1803.10122>.
- [9] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640:647–653, 2025. doi: 10.1038/s41586-025-08744-2. URL <https://doi.org/10.1038/s41586-025-08744-2>.
- [10] Hugging Face Biology Research. Carbon-500M model card. Hugging Face model card, 2025. URL <https://huggingface.co/HuggingFaceBio/Carbon-500M>. 500M-parameter decoder-only autoregressive DNA model, Carbon family, Apache-2.0.
- [11] Yanrong Ji, Zhihan Zhou, Han Liu, and Ramana V. Davuluri. DNABERT: pre-trained bidirectional encoder representations from transformers model for DNA-language in genome. *Bioinformatics*, 37(15):2112–2120, 2021. doi: 10.1093/bioinformatics/btab083. URL <https://doi.org/10.1093/bioinformatics/btab083>.
- [12] Konrad J. Karczewski, Laurent C. Francioli, Grace Tiao, Beryl B. Cummings, Jessica Alfoldi, Qingbo Wang, Ryan L. Collins, Kristen M. Laricchia, Andrea Ganna, Daniel P. Birnbaum, et al. The mutational constraint spectrum quantified from variation in 141,456 humans. *Nature*, 581:434–443, 2020. doi: 10.1038/s41586-020-2308-7. URL <https://doi.org/10.1038/s41586-020-2308-7>.
- [13] Melissa J. Landrum, Jennifer M. Lee, George R. Riley, Wonhee Jang, Wendy S. Rubinstein, Deanna M. Church, and Donna R. Maglott. ClinVar: public archive of relationships among sequence variation and human phenotype. *Nucleic Acids Research*, 42(D1):D980–D985, 2014. doi: 10.1093/nar/gkt1113. URL <https://doi.org/10.1093/nar/gkt1113>.
- [14] Eric Nguyen, Michael Poli, Marjan Faizi, Armin W. Thomas, Callum Birch-Sykes, Michael Wornow, Aman Patel, Clayton Rabideau, Stefano Massaroli, Yoshua Bengio, Stefano Ermon, Christopher Ré, and Stephen Baccus. HyenaDNA: Long-range genomic sequence modeling at single nucleotide resolution. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2306.15794>.
- [15] Eric Nguyen, Michael Poli, Matthew G. Durrant, Brian Kang, Dhruva Katrekar, David B. Li, Liam J. Bartie, Armin W. Thomas, Samuel H. King, Garyk Brixi, Jeremy Sullivan, Madelena Y. Ng, Ashley Lewis, Aaron Lou, Stefano Ermon, Stephen A. Baccus, Tina Hernandez-Boussard, Christopher Ré, Patrick D. Hsu, and Brian L. Hie. Sequence modeling and design from molecular to genome scale with evo. *Science*, 386(6723):ead9336, 2024. doi: 10.1126/science.ado9336. URL <https://doi.org/10.1126/science.ado9336>.
- [16] Yair Schiff, Chia-Hsiang Kao, Aaron Gokaslan, Tri Dao, Albert Gu, and Volodymyr Kuleshov. Caduceus: Bi-directional equivariant long-range DNA sequence modeling. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL <https://arxiv.org/abs/2403.03234>.

A Notation

Symbol	Meaning
$w_{\text{ref}}, w_{\text{alt}}$	reference / edited DNA window (12,288 bp = 2,048 six-mer tokens)
$\text{enc}(\cdot)$	frozen Carbon-500M encoder (layer 20, centered-mean pool $r=8$, L_2 -normalized)
$s_t = \text{enc}(w_{\text{ref}})$	reference latent state, $\mathbb{R}^{1024}$
$s_{t+1} = \text{enc}(w_{\text{alt}})$	true post-edit state (training target / eval ground truth)
$\hat{s}_{t+1} = g(s_t, a)$	predicted post-edit state (residual, eq. (2))
$\Delta = s_{t+1} - s_t$	true edit residual (small norm; core of section 6)
a, a_{emb}	edit action and its embedding, $\mathbb{R}^{64}$
g	cross-attention residual predictor (6 cross + 2 self, $d_{\text{hidden}}=768$)
α, β	prediction-loss weights (1.0, 0.1); $\gamma=0.5$ Phase-2 KL weight
$\sigma_{\text{raw}}, \sigma_{\text{cal}}$	raw / context-calibrated surprise (165 buckets)
$\text{score}_{\text{Carbon}}$	$\log P(w_{\text{ref}}) - \log P(w_{\text{alt}})$ (zero-shot baseline)
N	evaluated variants per slice: 16/16/32/32/8/8

Table 6: **Content-addressed artifact identity for the evaluated release.** Every reported number is bound to these identities through checksum receipts; the benchmark suite re-renders its report (and this manuscript’s machine-generated companion) from the artifacts and rejects stale text or mutated evidence (§9). All 28 suite outputs verified.

Field	Value
Model release	geno-lewm-v0.2.1-r1
Model id	sha256:cddb8f3b9671...f5f3ed6af4
Dataset snapshot	geno-lewm-data-v0.2.1-r1
Commit	d9b06815cf8e...5aa4154d
Hardware	NVIDIA H200 (Linux x86_64, glibc 2.35)
Suite outputs	28 / 28 verified (release_inputs: pass)

B Artifact Identity

C Citation Verification Notes

All bibliography entries were verified programmatically against publisher pages, DOIs, or arXiv. Two caveats apply. The Carbon-500M entry uses an organizational author because the model card exposes no formal author list or technical-report citation. The BRCA2 saturation genome-editing assay used as a continuous-label benchmark is described generically and *not* attributed to a specific publication here, because we could not verify the exact source; we deliberately avoid a plausible-but-wrong citation (the widely cited Findlay et al. saturation genome-editing study concerns BRCA1, not BRCA2). A small number of page numbers in otherwise-verified entries are inferred and flagged in the bibliography file.